

DRL-OR: Deep Reinforcement Learning-based Online Routing for Multi-type Service Requirements

Chenyi Liu^{*‡§}, Mingwei Xu^{*†‡§}, Yuan Yang^{*‡§}, Nan Geng^{*‡§},

^{*}Department of Computer Science and Technology, Tsinghua University, Beijing, China

[†]Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China

[‡]Beijing National Research Center for Information Science and Technology (BNRist), Beijing, China

[§]Peng Cheng Laboratory (PCL), Shenzhen, China

liuchenyi19@mails.tsinghua.edu.cn, xumw@tsinghua.edu.cn, yangyuan_thu@mail.tsinghua.edu.cn, gn16@mails.tsinghua.edu.cn

Abstract—Emerging applications raise critical QoS requirements for the Internet. The improvements of flow classification technologies, software defined networks (SDN), and programmable network devices make it possible to fast identify users' requirements and control the routing for fine-grained traffic flows. Meanwhile, the problem of optimizing the forwarding paths for traffic flows with multiple QoS requirements in an online fashion is not addressed sufficiently. To address the problem, we propose DRL-OR, an online routing algorithm using multi-agent deep reinforcement learning. DRL-OR organizes the agents to generate routes in a hop-by-hop manner, which inherently has good scalability. It adopts a comprehensive reward function, an efficient learning algorithm, and a novel deep neural network structure to learn an appropriate routing policy for different types of flow requirements. To guarantee the reliability and accelerate the online learning process, we further introduce safe learning mechanism to DRL-OR. We implement DRL-OR under SDN architecture and conduct Mininet-based experiments by using real network topologies and traffic traces. The results validate that DRL-OR can well satisfy the requirements of latency-sensitive, throughput-sensitive, latency-throughput-sensitive, and latency-loss-sensitive flows at the same time, while exhibiting great adaptiveness and reliability under the scenarios of link failure, traffic change, and partial deployment.

I. INTRODUCTION

There has been a rapid growth of brand new applications in recent years, which not only imposes higher and higher requirements on communication technologies such as 5G, but also brings great challenges to the Internet. In particular, each application has unique yet stringent requirements on latency, jitter, throughput, and packet loss rate, etc. For example, HD video streaming needs high throughput, live video and remote VR desire low latency and high throughput, while online gaming service is sensitive to latency and packet loss. Unfortunately, traditional best-effort forwarding of the Internet can hardly satisfy such requirements due to congestions, long path lengths, and lossy links.

A large body of literature has been devoted to differentiated service [1] [2] and quality of service (QoS) routing [3]–[6]. Recently, the improvements of flow classification technologies [7] [8], software defined networks (SDN) [9], and programmable network devices [10] [11] make it possible to

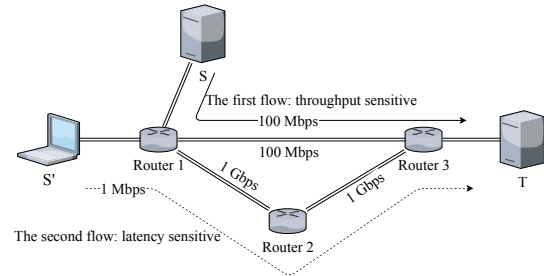


Fig. 1: A non-optimal routing decision caused by not considering the requirements of upcoming flows.

fast identify user requirements and control the routing for fine-grained traffic flows. However, how to optimize the routing efficiently to meet varied QoS requirements of different applications remains an open problem. Most existing approaches [3] model the problem as optimization models, and solve them with optimization methods. Such approaches assume that the network status is relatively static during a period. Nevertheless, user flows with different QoS requirements enter and quit a network frequently, and routing is essentially an online decision making problem. In such a case future dynamics should be sufficiently considered, including not only arriving and leaving of flows, but also topology change such as link failures and network upgrades. A routing decision without considering the requirements of upcoming flows may incur degradation on network performance. Fig. 1 shows an example. The first flow from node S to node T has a data rate of 100 Mbps and is throughput-sensitive, for which a greedy algorithm chooses path S -1-3- T . Then, the second flow arrives, which has a data rate of 1 Mbps from node S' to T , and is latency-sensitive. The greedy algorithm has to choose path S' -1-2-3- T which is longer and has a large latency. This degrades the QoS of the second flow.

In this paper, we tackle the online routing optimization problem for multi-type service requirements by exploring the potential of deep reinforcement learning (DRL). DRL is expert at solving online decision making problems, which optimizes not only current but also future rewards and can adapt to the dynamic environment. There have been studies on rein-

forcement learning-based routing [12]–[17]. However, most existing studies treat all flows as the same and do not consider differentiated QoS requirements. To address the problem, we have to overcome a set of challenges. First, the flows with different QoS requirements are not independent, and they share limited network resources. The routing decisions need to consider all requirements concurrently, while preserving the capability to satisfy future requirements. Second, the approach should be scalable to network size, flow number, as well as service type. That is to say, unknown QoS requirements that may arise in the future should also be supported. Last but not least, the route should be stable and will not change from time to time, even in a dynamic environment. We restrict that once a path for a flow is computed, the path will never change unless there is a link failure in the path. Meanwhile, the path computed for newly coming flows should adapt to new network status including burst traffic and link failure.

We develop our scheme under the SDN architecture. However, using one single agent in a centralized node, e.g., a SDN controller, to make routing decisions may greatly harm the scalability of the scheme, because the action space dramatically increases with the size of network and will become too intractable to enable efficient training and accurate inference [16]. To address the issue, we decompose the route generation process into a multi-agent DRL problem. In particular, each router/switch is controlled by a DRL agent to select next hops which form routes, according to each flow destination address and QoS requirement. In such a scheme, the agents need to cooperate with each other to achieve the global optimal with limited messages exchanged, and unsafe routes (e.g., routing loops) due to random exploration by DRL should be avoided.

We propose DRL-OR, a multi-agent DRL-based online routing algorithm for multiple service requirements. We normalize the flow performance metrics such as latency and packet loss in a self-adaptive approach, and combine the normalized metrics to obtain utility functions for different service types. Based on the utility functions proposed, we model the route generation process as a multi-agent Markov decision process (MAMDP) and develop an efficient learning algorithm for DRL-OR scheme. To realize the DRL-OR agents, we propose a novel deep neural network (DNN) structure, which has a common feature extraction layer for all flows, and a specialized output layer for each service type and destination node. The proposed DNN structure can be easily scaled to accommodate new flow types and network update. The DRL agents keep learning online so as to adapt to network dynamics. To avoid unsafe routes, we propose to use offline pre-training and introduce the safe learning technique [18] for the online training process. The proposed reliable learning mechanisms accelerates the training process and provides a reliability guarantee for DRL-OR.

We conduct experimental evaluations on Mininet [19] using real-world network topologies and traffic traces. The results validate that DRL-OR with safe learning technique can learn a good routing policy to satisfy multi-type flow requirements and has great adaptiveness and reliability under link fail-

TABLE I: Service types

Service type	Typical application
Latency-sensitive	Web browsing
Throughput-sensitive	File transfer
Latency-throughput-sensitive	Live video
Latency-loss-sensitive	Online game

ure and traffic change. Compared with traditional rule-based routing algorithms, DRL-OR show a better performance in different network scenarios. Extra evaluation on incremental deployment validate that partially deploying DRL-OR can also improve flow performance significantly, which exhibits the flexibility and scalability of DRL-OR.

The remainder of the paper is organized as follows: We describe the overview of DRL-OR in Section II and present the design of DRL-OR concretely in Section III. Then, Section IV shows the experimental setup and the evaluation results. Finally, we provide a brief overview of related work in Section V and conclude the paper in Section VI.

II. OVERVIEW

We consider a network where any node may originate a flow terminated at another node at any time. Each flow has certain requirements on a set of performance metrics, including latency, throughput, and packet loss rate. The requirements can be clustered into a few categories, namely service types. We specify four service types shown in Table I. For each service type we propose a utility function. The utility of a flow reflects to what extent the flow's requirements are satisfied. Our target is to compute the next hop for each flow at each node traversed to generate a route, such that 1) each flow can be forwarded from the source node to the destination node successfully, i.e., there is no routing loop or blackhole; and 2) the total utility of all flows is maximized.

Formally, the network is modeled as directed graph $G(V, E)$, where node set V represents the collection of switches/routers, and edge set E represents the collection of links. Each edge has a finite capacity. A flow request emerges in the network at some time point with a fixed bandwidth demand. Without loss of generality, we assume that the flow requests arrive in sequence. Recall that we restrict the route of a flow to be stable once computed. Thus, the network status will remain stable until the next flow arrives or some existing flows are finished. For the convenience of modeling, we assume a discrete-time model where the time is divided into contiguous timeslots $t = 1, 2, \dots$, and the t -th flow request arrives at the beginning of timeslot t .¹ Let src_t be the source node of the t -th flow and dst_t be the destination node. Let K be the number of service types and η_t be the service type of the t -th flow ($1 \leq \eta_t \leq K$). Let $U_{\eta_t}(\cdot)$ be the utility function of the service type η_t , so $U_{\eta_t}(t)$ is the utility of the t -th flow. The objective of our problem is then

¹Generally, we assume that the interval between flow arrivals is greater than the time for the network to install a new route, so it is reasonable to assume that multiple flow requests will not arrive in the same timeslot. In reality, we can also filter the flow requests to be handled, so as to enlarge the interval between flow arrivals.

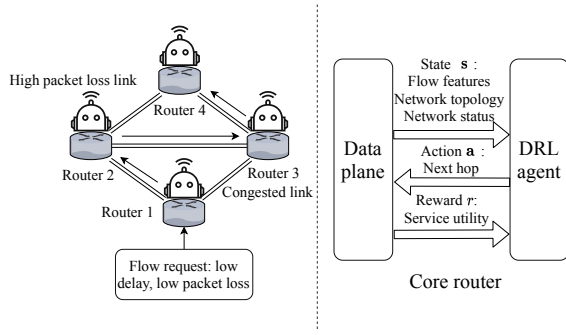


Fig. 2: Deep reinforcement learning-based online routing scheme for multi-type service requirements (DRL-OR).

$$\text{maximize } \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T U_{\eta_t}(t). \quad (1)$$

We propose a multi-agent DRL-based online routing scheme, as shown in Fig. 2. Instead of using only one agent for the whole network, which results in an exponential action space that greatly increases the complexity and degrades the accuracy of routing decision inference [16], our scheme deploys one DRL agent for each node. The agent just needs to decide the next hop of an upcoming flow to form a route. In particular, an agent makes a fast routing decision inference on the arrival of a new flow request, and the results are downloaded into local data plane of the router, so the packets can be forwarded accordingly. Such scheme can be realized with help of SDN architecture where the DRL agents are placed in a centralized controller and control routing nodes as described before.

A good routing decision is made based on proper inputs, i.e., the agent input state. First, a few features of current flow request are needed, including source node src_t , destination node dst_t , service type η_t , and the maximum data rate $demand_t$. Second, we need the network topology as well as link attributes (e.g., capacities and loss rates), which are relatively static and can be obtained once for all, unless there is a topology change such as link failures. Third, we also need current network status, e.g., the residual capacity of each link which reflects the congestion degree. In reality, link residual capacity can be obtained periodically. In an ideal model, we assume that the information can be obtained instantly after a flow request arrives, i.e., in each timeslot.

Unlike traditional model-based solutions, our multi-agent DRL routing scheme allows each agent to learn online, so as to adapt to dynamically changing network environment. To this end, a feedback is returned to the agents after an action (routing decision) is taken. In the field of DRL, this feedback is called a reward. In particular, when agent i observes state s_t^i for the t -th flow, action a_t^i is taken and reward r_t is returned. In general, the objective of DRL is to maximize the discounted cumulative reward $\sum_{t=1}^T \gamma^t r_t$, where $\gamma \in [0, 1]$ is the discount factor. In this paper, reward r_t is the utility value of the t -th flow. We assume that the performance metrics of a flow can be collected to compute the utility, which is returned to each

agent after the routing decision is made. In IV we show that DRL-OR can be deployed under SDN architecture. But with the highly distributed multi-agent DRL scheme, DRL-OR also has great potential to embrace distributed deployment.

III. THE DRL-OR SCHEME

In this section, we describe the details of the DRL-based online routing algorithm for multi-type service requirements (DRL-OR) scheme. First, we present the utility function to evaluate the quality of routing paths for different types of flows. Next, we model the route generation process as a multi-agent Markov decision process and present the learning algorithms of agents. Then, we show a novel neural network structure for approximating the policy functions of agents. After that, the safe learning mechanism is introduced to improve the reliability of our system. Finally, we give a discussion on the incremental deployment of DRL-OR.

A. Utility Function

To evaluate the routing paths for flows with different service types, we need to tailor the utility functions for flows properly. We propose a general utility function for flows with various service types by combining multiple performance metrics while fairness among flows is also considered. In particular, we consider three performance metrics including throughput, latency, and packet loss ratio. Let x_t , y_t , and z_t be the throughput value, latency value, and packet loss ratio of the t -th flow, respectively. Our proposed utility function is then

$$U_t = w_1 \log x_t - w_2 y_t^{\alpha_1} - w_3 z_t^{\alpha_2}, \quad (2)$$

where w_1 , w_2 , and w_3 are nonnegative scalars and indicate the importance weight of the performance metrics. To enhance the fairness among flows, we apply function $\log(\cdot)$ to throughput and function $(\cdot)^{\alpha_1}$ and $(\cdot)^{\alpha_2}$ ($\alpha_1, \alpha_2 \geq 1$) to latency and packet loss ratio, respectively. Such a fairness-enhanced utility function is in line with many previous works like [15] [20]. In this paper, we set $\alpha_1 = 1$ and $\alpha_2 = 1$. For latency-sensitive flows, w_1 , w_2 , and w_3 are 0, 1, and 0 respectively. For throughput-sensitive flows, w_1 , w_2 , and w_3 are 1, 0, and 0 respectively. For latency-throughput-sensitive flows, w_1 , w_2 , and w_3 are 0.5, 0.5, and 0 respectively. For latency-loss-sensitive flows, w_1 , w_2 , and w_3 are 0, 0.5, and 0.5 respectively.

However, collected raw data of performance metrics (i.e., x_t , y_t , and z_t) cannot be input into the utility function directly. As we all know, different performance metrics such as throughput and latency have very different scales. Further, in real networks, the same performance metric (e.g., latency) depends on many factors (e.g., the traffic load of networks) and may change greatly with time. It is not easy to adjust the weights (i.e., w_1 , w_2 , and w_3) to combine different performance metrics properly. As a result, the scale of computed utility values will also change greatly with time, which makes learning algorithms difficult to converge well. To solve the problem, we normalize the raw performance metrics before inputting them into the utility function.

Particularly, the throughput value of a flow can be directly normalized by the flow's maximum data rate $demand_t$. We define throughput ratio as the raw throughput value divided by the corresponding $demand_t$. For the other two performance metrics (i.e., latency and packet loss ratio), we apply a self-adaptive mechanism to do normalization. We normalize the raw performance metric value of a target flow by an average metric value. Let q_t ($q_t \in \{y_t, z_t\}$) be the raw performance metric value of the t -th flow and $\hat{q}_t$ be the normalized value. Formally, q_t can be normalized by

$$\hat{q}_t = \frac{q_t}{\bar{q}_t}, \quad (3)$$

where $\bar{q}_t$ denotes the average performance metric value of the recent flows with the same source and destination node as the t -th flow. For convenience, $\bar{q}_t$ is approximated by

$$\bar{q}_{t+1} = \varepsilon \cdot \bar{q}_t + (1 - \varepsilon) \cdot q_t, \quad (4)$$

where $\varepsilon \in [0, 1]$ is a constant parameter controlling the update speed of $\bar{q}_t$, and we use $\varepsilon = 0.99$.

B. Multi-agent Learning Algorithm

As described previously, we compute a route for each flow in a hop-by-hop manner to decompose the route problem into several sub-problems with much smaller state and action spaces. When a flow request reaches a node, the node (also the agent) makes an action to select the best next hop for the flow. The following nodes take a similar way until the flow request reaches the destination node. Finally, we obtain the route for the flow (i.e., a sequence of ordered nodes).

We model the route generation process as a typical MAMDP where each node is an agent. The MAMDP is characterized by a tuple $(\mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R})$. $\mathcal{S}$ is the global state space shared by all the agents $i \in \mathcal{N}$, and $\mathcal{A}^i$ denotes the action space of agent i . The joint action space of all agents is denoted as $\mathcal{A} = \prod_{i \in \mathcal{N}} \mathcal{A}^i$. Then, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition probability of the MAMDP, and $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the function for computing global rewards. We take the utility function of Eq. (2) as the reward function. Note that, the state space and reward values can be observed globally by each agent in such a MAMDP.

Now, the route of a flow can be determined by action $\mathbf{a} \in \mathcal{A}$ after all the agents take a local action based on current state $\mathbf{s} \in \mathcal{S}$ (for notational simplicity, we omit the subscript t). Actually, only the agents on the path need to make a decision, and the decision making of an agent depends on the decisions made by the prior agents in the same path. Assume that $\{src, \mathbf{a}^1, \dots, \mathbf{a}^{n-1}, dst\}$ is a route determined by action $\mathbf{a}$. Then, the process of making action $\mathbf{a}$ based on state $\mathbf{s}$ can be expressed as

$$Pr(\mathbf{a}|\mathbf{s}) = Pr(\mathbf{a}^1|\mathbf{s})Pr(\mathbf{a}^2|\mathbf{s}, \mathbf{a}^1)Pr(\mathbf{a}^3|\mathbf{s}, \mathbf{a}^1, \mathbf{a}^2) \dots Pr(\mathbf{a}^n|\mathbf{s}, \mathbf{a}^1, \mathbf{a}^2, \dots, \mathbf{a}^{n-1}), \quad (5)$$

where $\mathbf{a}^i$ denotes the i -th agent's action which determines the $(i+1)$ -th node of the routing path.

To describe the relations between the local action $\mathbf{a}^i$ and actions before $\mathbf{a}^i$, we import *conditional state* as an addition to global state $\mathbf{s}$. Particularly, conditional state of an agent records the currently used nodes before the current node with respect to a specific flow. In our design, we take the form of a zero-one vector with length $|V|$ for conditional state, i.e., the i -th item in the vector is 1 if the i -th node has been selected to construct the routing path, otherwise 0. When making an action for a coming flow request, the agent takes the corresponding conditional state together with the global state as the actual input.

Considering that the state space and the action space in real networks are very large, we adopt neural networks to approximate policies of agents, which maps state input to action output. Formally, the local policy network of agent i can be denoted as $\pi_{\theta^i}(\mathbf{a}^i|\mathbf{s}, \mathbf{c}^i)$, where θ^i is the corresponding parameters of the neural network and $\mathbf{c}^i$ is the conditional state. Essentially, $\pi_{\theta^i}(\mathbf{a}^i|\mathbf{s}, \mathbf{c}^i)$ is used to approximate the target conditional probability distribution $Pr(\mathbf{a}^i|\mathbf{s}, \mathbf{a}^1, \mathbf{a}^2, \dots, \mathbf{a}^{i-1})$.

In our MAMDP model, the agents need to collaboratively find a policy π_{θ^i} that maximizes the globally averaged long-term reward over the network routing problem. Let $J(\theta)$ be the global objective that needs to be maximized, where θ is the collection of every agent's policy parameters θ^i and can be updated by computing the gradient of $J(\theta)$.

The most commonly used policy gradient estimator for reinforcement learning [21] has the form

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [A(\mathbf{s}, \mathbf{a}) \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s})], \quad (6)$$

where τ denotes state-action pair $(\mathbf{s}, \mathbf{a})$ sampled through $\tau \sim p_{\theta}(\tau)$ with respect to π_{θ} and $A(\mathbf{s}, \mathbf{a})$ is an advantage value estimator. $A(\mathbf{s}, \mathbf{a})$ evaluates how much better taking action $\mathbf{a}$ is than taking actions randomly under policy π_{θ} with respect to state $\mathbf{s}$ [22]. Particularly, a positive (*resp.* negative) value of $A(\mathbf{s}, \mathbf{a})$ means taking action $\mathbf{a}$ is better (*resp.* worse) than randomly taking actions with respect to current policy π_{θ} .

According to Eq. (5) and Eq. (6), we can derive the policy gradient estimator for each agent's policy parameters as

$$\nabla_{\theta^i} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [m^i A(\mathbf{s}, \mathbf{a}) \nabla_{\theta^i} \log \pi_{\theta^i}(\mathbf{a}^i|\mathbf{s}, \mathbf{c}^i)], \quad (7)$$

$$m^i = \begin{cases} 1, & \text{if node } i \text{ on the path,} \\ 0, & \text{if node } i \text{ not on the path.} \end{cases}$$

Compared with policy gradient estimator in Eq. (6), a mask variable m^i is used in Eq. (7) to indicate whether node i is on the path. $m^i = 0$ will result in $\nabla_{\theta^i} J(\theta) = 0$, which means the node i won't update its policy parameters θ^i at this step.

Agents can update policy parameters using the gradients computed by Eq. (7) in the same way to typical policy gradient methods. However, policy gradient methods are sensitive to the update step size selection, which makes it challenging to obtain good results. Besides, they often have very poor sample efficiency and take millions of time steps even for simple tasks. In this paper, we adopt the Proximal Policy Optimization (PPO) algorithm [21], one of the most popular reinforcement learning approaches. In contrary to policy gradient methods,

PPO has been proved to have better data efficiency and low training variance. We introduce how to expand PPO to the multi-agent reinforcement learning framework of DRL-OR below.

First, PPO reduces sample complexity via so-termed importance sampling. Unlike policy gradient methods that sample state-action pair $\tau \sim p_\theta(\tau)$ using the newest policy parameters θ , PPO efficiently uses samples based on old policy parameters θ_{old} to obtain a better data efficiency. By assuming that state distributions under θ and θ_{old} are similar, we can derive the gradient estimator of agent i as

$$\nabla_{\theta^i} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta_{old}}(\tau)} [r(\theta^i) m^i A(\mathbf{s}, \mathbf{a}) \nabla_{\theta^i} \log \pi_{\theta^i}(\mathbf{a}^i | \mathbf{s}, \mathbf{c}^i)], \quad (8)$$

where $r(\theta^i) = \frac{\pi_{\theta^i}(\mathbf{a}^i | \mathbf{s}, \mathbf{c}^i)}{\pi_{\theta_{old}}(\mathbf{a}^i | \mathbf{s}, \mathbf{c}^i)}$. We can find state-action pair τ is sampled through $\tau \sim p_{\theta_{old}}(\tau)$. $r(\theta^i)$ is the importance weight of samples, i.e., the ratio of the probability under the new and old policies. In the case where θ_{old}^i equals θ^i , $r(\theta^i)$ becomes 1. The corresponding local objective function of Eq. (8) has the form

$$J(\theta^i) = \mathbb{E}_{\tau \sim p_{\theta_{old}}(\tau)} [m^i A(\mathbf{s}, \mathbf{a}) r(\theta^i)]. \quad (9)$$

Note that, the distribution of $\pi_{\theta^i}(\mathbf{a}^i | \mathbf{s}, \mathbf{c}^i)$ should not be too different from $\pi_{\theta_{old}}(\mathbf{a}^i | \mathbf{s}, \mathbf{c}^i)$. Otherwise, we need to take a large number of sample processes so as to have a good knowledge of current $J(\theta^i)$.

Second, PPO limits the difference between θ^i and θ_{old}^i to make θ^i not update too fast. Following PPO, we add a clip value function $\text{clip}(\cdot, \cdot, \cdot)$ and a min function $\min(\cdot, \cdot)$ to Eq. (9). Then, the new local objective function becomes

$$J^{CLIP}(\theta^i) = \mathbb{E}_{\tau \sim p_{\theta_{old}}(\tau)} [m^i \cdot \min(A(\mathbf{s}, \mathbf{a}) r(\theta^i), A(\mathbf{s}, \mathbf{a}) r'(\theta^i))], \quad (10)$$

where $r'(\theta^i) = \text{clip}(r(\theta^i), 1 - \epsilon, 1 + \epsilon)$ and ϵ is a hyperparameter, set to 0.1 in this paper. Using the gradients of Eq. (10), the probability of a good action with $A^i(\mathbf{s}, \mathbf{a}) > 0$ won't increase too much and the probability of a bad action with $A^i(\mathbf{s}, \mathbf{a}) < 0$ won't decrease too much. In this way the agents can update their routing policy in a consistent and stable speed. Note that Eq. (10) is easy to be optimized by gradient decent algorithms.

In the above analysis, global state is used for decision making (i.e., one of the inputs of π_{θ^i}) as well as the advantage value estimator (i.e., $A_\theta^i(\mathbf{s}, \mathbf{a})$). However, for the routing problem in this paper, global state is insufficient for an agent to select the best next hop. Particularly, processed information of neighbor nodes (e.g., the distances and available bandwidth from the neighbor nodes to the destination node) can help an agent select the best next hop, but combining these information into a global state input will induce high computational overhead and make the policy learning algorithm of agents difficult to converge especially in large networks. Thus, we use partial state for each agent (We describe partial state concretely in Section III-C). The policy function of agent

i can be denoted as $\pi_{\theta^i}(\mathbf{a}^i | \mathbf{s}^i, \mathbf{c}^i)$, where $\mathbf{s}^i$ represents the partial state of agent i . Then, the advantage value estimator becomes $A^i(\mathbf{s}^i, \mathbf{a})$ using partial state $\mathbf{s}^i$. Since flow utilities are broadcast to all the agents and the partial state $\mathbf{s}^i$ contains enough network status, we can expect an unbiased estimation $A^i(\mathbf{s}^i, \mathbf{a})$ locally to advantage value $A(\mathbf{s}, \mathbf{a})$ which uses the generalized advantage estimation [23]. Note that with local advantage value estimator $A^i(\mathbf{s}^i, \mathbf{a})$ the scheme have better potential for distributed deployment.

C. Policy Network Design

In DRL-OR, policy function is approximated by a deep neural network (DNN) and the parameters of the DNN can be updated using gradient estimator described above. In this subsection, we present how the policy network maps state input to action output for a coming flow request in detail. First, we present the design of state input (including partial state $\mathbf{s}^i$ and conditional state $\mathbf{c}^i$) and action output (i.e., $\mathbf{a}^i$) of a general agent i , and then show a novel policy network structure.

State input: The state input of the policy network acts as a vector which consists of partial state $\mathbf{s}^i$ and conditional state $\mathbf{c}^i$. Particularly, partial state contains flow features (including the source node, the destination node, the service type, and the maximum data rate), topology information (including the neighbors' shortest path distances to different destinations and the packet loss ratios of links), and resource usage information (including links' residual capacity and neighbors' maximum available bandwidth to different destinations). Conditional state is a zero-one vector with length $\mathcal{N}$, which indicates which nodes have been chosen to construct the routing path of the coming flow request as described previously. Note that we add processed neighbor nodes information to state input to help the neural network make better routing decisions.

Action output: The action output for a coming flow request is also a vector which is the probability distribution of selecting neighbor nodes as the next hop.

Policy network structure: To map state input to action output, a direct design is to use one deep neural network as the policy function for all types of flows (e.g., flows with different service types or different source/destination nodes). However, the optimal policy parameters of different types of flows can be in high diversity. Using a set of completely shared policy parameters for different types of flows will lead to conflicts of parameter updates especially when the network topology is large. In such a case, the policy network is difficult to converge. An alternative approach to solve the problem is using a specialized policy neural network for every type of flows, which, however, results in a huge scale of policy parameters and significantly slow convergence.

We propose a partially unique policy network structure to deal with the high diversity among the optimal policies of flows while still has a fast model convergence. The novel policy network structure consists of three parts. The first part is one input layer, which includes $O(|V|^2)$ neurons. The second part is two general feature extraction layers, where each layer

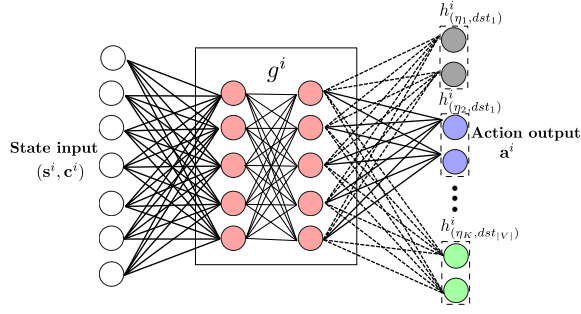


Fig. 3: The partially unique policy network structure.

has 64 neurons. The third part is a group of specialized policy layers for different types of flow requests. The DNN takes the original state as input and extracts the feature vector. Then, the extracted features are fed into the specialized policy layer corresponded to its destination and service type. The specialized policy layer will then output the action for the corresponding type of flows. The proposed policy network for agent i can be formally expressed as

$$\pi_{\theta^i}(\mathbf{a}^i | \mathbf{s}^i, \mathbf{c}^i) = h^i_{(\eta, dst)}(\mathbf{a}^i | g^i(\mathbf{s}^i, \mathbf{c}^i)), \quad (11)$$

where g^i denotes general feature extraction layers shared by all types of flows and $h^i_{(\eta, dst)}$ denotes the specialized policy layer for the flow requests with service type η and destination node dst . Fig. 3 shows the partially unique policy network structure and its working process when a flow request with service type η_2 and destination node dst_1 comes to agent i .

We note that the proposed policy network structure has good flexibility. A new type of flow requests can be accommodated by just adding one more specialized policy layer. Current well-trained feature extraction layers make the parameters of the newly added layer be optimized quickly.

D. Reliable Learning Mechanisms

It is well known that reinforcement learning takes an exploration-and-exploitation method during the learning stage. That is to say, agents may try some exploration actions randomly so as to obtain better policy parameters. Doing explorations in real networks may cause unsafe routing decisions (e.g., routing loops) and thus leads to terrible routing performance. To enhance the reliability of our system, we propose two mechanisms.

A DRL agent learning from scratch usually needs thousands of iterations to learn a stable policy and a multi-agent system makes the learning process even longer. To reduce the possible performance loss during the initial learning stage, we pre-train agents offline before deploying them in real networks. In the offline pre-training stage, the agents are guided to learn the shortest path routing rule. After being deployed in real networks, the agents take online learning and converge faster based on the prior knowledge obtained by pre-training.

The pre-training stage makes our system converge faster online, but it cannot guarantee the reliability of our system. Unsafe routing decisions like routing loops can still happen due to action explorations or unexpected changes of the

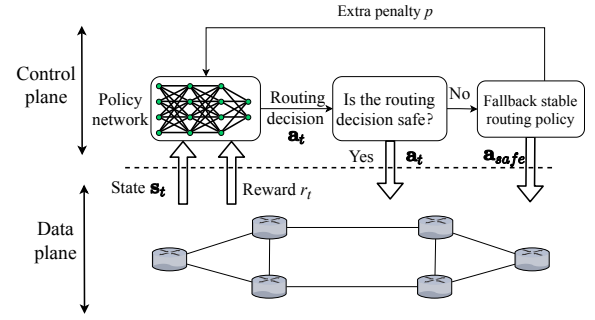


Fig. 4: An illustration of the safe learning mechanism.

network environment. Inspired by [18], we propose a safe learning mechanism to avoid unsafe routings.

Fig 4 illustrates our proposed safe learning mechanism. After an action is made, a safe learning component will check whether the generated routing path is a legal path (i.e., connecting the source node and the destination node without loops and heavily congested links). If the routing path is safe, it will be enabled in the data plane directly. If not, the safe learning component will turn to the fallback policy to generate a legal routing path for the flow request. Particularly, the fallback policy computes the shortest path that fulfilling the maximum data rate requirement of the flow request. If there is no such a satisfactory path, the fallback policy chooses the path with the largest available bandwidth. After the path computation of the fallback policy, the routing nodes will be informed of the new routing rules. To reinforce the experience of unsafe action output by agents, an extra penalty value will be added to the reward. In this way, the reliability in the online learning stage is guaranteed, and our system model can still converge to the optimal policy.

E. Incremental Deployment

In practice, deploying the system of DRL-OR requires both software and hardware updates on routers. Thus, deployment on a running real network at once may be difficult and introduces much overhead. Alternatively, we can deploy DRL-OR incrementally, which benefits from our distributed system design. Some key routers can be controlled by DRL agents, and the other routers execute original routing algorithms (e.g., OSPF). In this way the remaining problem is to find the minimal key router set to achieve maximum network performance improvement. Existing incremental deployment approaches [24] [25] [26] can be easily applied in our scenario. In IV we show a simple example of incremental deploying DRL-OR on real-world network topologies.

IV. PERFORMANCE EVALUATION

A. Experiment Setup

We use SDN architecture for the convenience of constructing the experimental network and implementing the approaches for comparison. We construct an experimental network using Ryu [27] and Mininet [19] in a server with 6-core 3.7 GHz CPU and 8 GB memory. DRL-OR is implemented

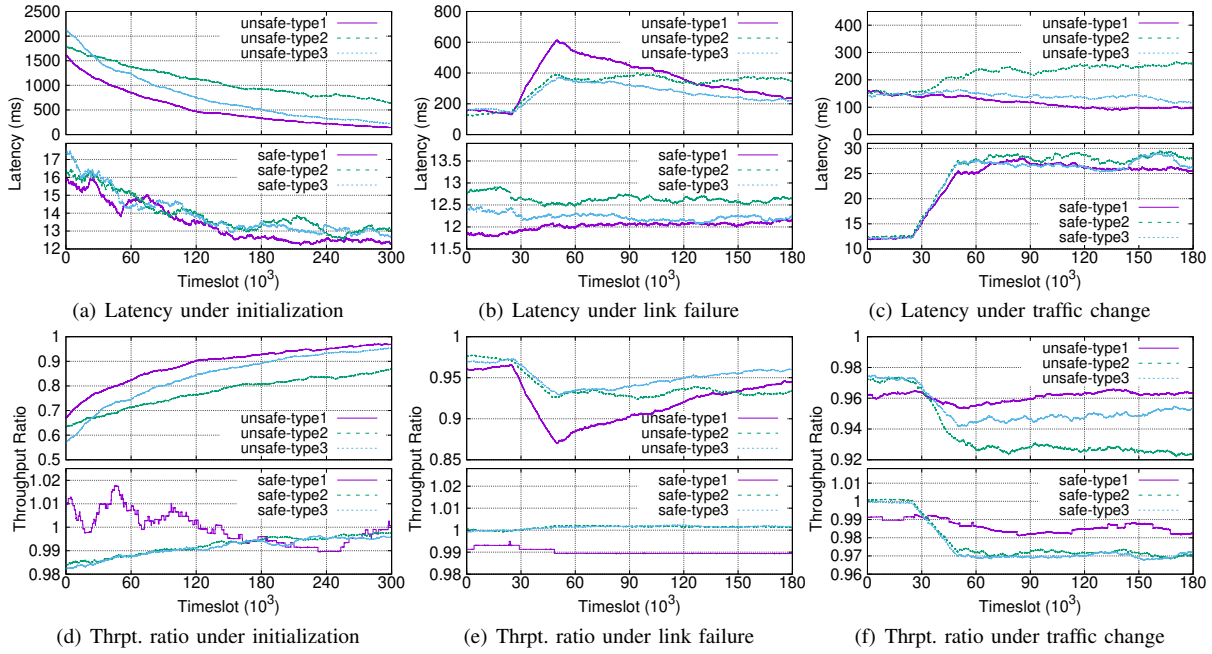


Fig. 5: Results of DRL-OR on flow latency and throughput ratio under initialization, link failure and traffic change in Abilene.

with Pytorch 1.0 based on PPO algorithm [28]. The source code of DRL-OR is available at <https://github.com/netlab-lcy/DRL-OR>. In our experiments, an agent needs only 0.8 ms to select a next hop given an input state.

We use two real-world network topologies, i.e., Abilene [29] and GEANT [30]. Abilene is a topology which has 11 nodes and 14 bidirectional links. GEANT is a larger topology with 23 nodes and 37 bidirectional links. Since the data rate of the platform is limited, the link capacities are set smaller than reality. In particular, most links have a data rate of 10 Mbps, while one bottleneck link in Abilene and two in GEANT have a data rate of 2.5 Mbps. Each link has a latency of 5 ms.

The flow requests are generated randomly based on real traffic matrices measured by [29] [30]. In particular, the probability of selecting a source-destination pair is proportional to the real traffic volume between this source-destination pair. We use two sets of probabilities to determine the service type of a flow. In the first case, a flow is latency-sensitive (type I) with a probability of 0.4, throughput-sensitive (type II) with a probability of 0.3, and latency-throughput-sensitive (type III) with a probability of 0.3, and there is no latency-loss-sensitive (type IV) flow. In the second case, a flow is type I, type II, type III, and type IV with a probability of 0.2, 0.3, 0.3, and 0.2, respectively. The sending rates of type I, type II, type III, and type IV flows are fixed to 100 Kbps, 1500 Kbps, 1500 Kbps, and 500 Kbps, respectively. For Abilene we specify a light-load scenario where the flow duration is 10 timeslots, and a heavy-load scenario where the flow duration is 50 timeslots. For GEANT we set the flow duration to 15 timeslots.

We choose three typical routing algorithms for comparison. First, we evaluate the shortest path routing (SPR), which always uses the route with the least hop number for each

flow request. Second, we evaluate a load balancing routing (LBR), which uses the route with the largest residual capacity. Third, we evaluate a QoS routing (QoSR). QoSR uses SPR for a latency-sensitive flow, uses LBR for a throughput-sensitive flow, and computes a capacity-constrained shortest path [31] for a latency-throughput-sensitive flow. Since it is difficult to optimize network packet loss composed of both random packet loss on links and congestion packet loss on the router, QoSR uses SPR for a latency-loss-sensitive flow. We evaluate latency, throughput ratio, and packet loss ratio of each service type when each approach is used in our experiments.

We firstly evaluate the system convergence and the reliability of DRL-OR under situations of initialization, link failure and traffic change. Then we evaluate the effectiveness of DRL-OR compared with baseline routing algorithms in different network scenarios. Finally we evaluate the effectiveness of DRL-OR under partial deployment.

B. Convergence and Reliability

We evaluate the system convergence and reliability of DRL-OR under initialization process, link failure and traffic change. Fig. 5(a),(d) show the flow latency and flow throughput ratio results of DRL-OR under initialization process. The results of DRL-OR under link failure and traffic change are shown in Fig.5(b),(e) and Fig.5(c),(f), respectively.

1) *Initialization process*: We use the Abilene topology in the light-load scenario and assume three types of flow requests exist in the network (the first case as described in IV-A). The agents have been pretrained offline by shortest path routing policy but they still need to explore and learn a better routing policy, to adapt to current network environment and different QoS requirements. We see that the performance metrics of

TABLE II: Fallback policy trigger ratio (%)

	Initialization	Link failure	Traffic change
At the beginning	41.85	9.58	3.18
After convergence	2.30	2.39	2.73

3 types of flow shown in Fig. 5(a),(d) converge in a similar speed which reflects the online learning process of the agents. The throughput ratios achieve near 100% after convergence and latency-sensitive (type I) flows have the least latency. We also find that the latency is much less when the safe learning mechanisms developed in Section III-D are used (the curves tagged “safe”). This implies that the safe learning mechanism effectively accelerates the online learning process and avoids bad routing decisions incurring a large latency.

2) *Link failure*: We repeat the experiment using the trained model after the initialization process and injure a failure to the bottleneck link of Abilene at the 25,000-th timeslot. We observe that for DRL-OR without safe learning, the latency shown in Fig.5(b) increases sharply after the failure occurs, and the throughput ratio shown in Fig.5(e) decreases simultaneously. In contrary, when safe learning is used, the latency and the throughput ratio changes little. This is because the safe learning mechanism avoids the bad route generated by the agents based on history knowledge of the network and help the agents to learn a new policy in a short time.

3) *Traffic change*: Similarly, we switch the flow duration from 10 timeslots to 30 timeslots at the 25,000-th timeslot to simulate a traffic burst. In Fig.5(c) we observe that the latency increases but becomes stable soon when DRL-OR with safe learning is used. For DRL-OR without safe learning that has not converged well, the latency is very large and decreases slowly. In Fig.5(f), we see that DRL-OR either with or without safe learning experiences a throughput ratio decreasing after the traffic change, because more traffic has to be delivered. However DRL-OR with safe learning still provides better throughput for all of the three types of flow.

To further evaluate the influence of the the safe learning mechanism, we compute the trigger ratio of the fallback policy (i.e., the number of fallback policy trigger times divides the flow number in a period of time) in three cases, i.e., 1) during the initialization, 2) after the link failure, and 3) after the traffic change. Table II shows the results. We find that the fallback policy is triggered frequently at the beginning of online training, helping to generate safe route and adding a penalty to the agents. And when the model converges, unsafe route can be avoided with less usage of the fallback policy most of the time, even after link failure and traffic change.

C. Effectiveness Validation

We evaluate the performance metrics of DRL-OR in different network scenarios. To evaluate the adaptiveness of DRL-OR to packet loss, we add a 10% random packet loss to bottleneck links, and do experiments when all four types of flows are generated (the second case as described in IV-A). Table. III shows the results. We see that DRL-OR learns a near optimal routing policy in Abilene light-load scenario,

which achieves the best performance on latency, throughput and packet loss ratio. QoS-R also performances well on flow latency and throughput but causes much more packet loss for the latency-loss-sensitive flows (Type IV). LBR avoids using bottleneck links, leading to higher flow latency. SPR sometimes overloads bottleneck links, causing link congestion and leading to the worst performance. In Abilene heavy-load scenario, DRL-OR outperforms the three baselines in terms of every performance metric, which indicates that DRL-OR can adapt to complicated network environment. Note that in heavy-load scenario LBR is a better routing policy than QoS-R and SPR, but it is still beat by DRL-OR because such greedy-based methods do not consider the possible future flows.

To evaluate the generalization of DRL-OR, we also do the experiment on GEANT, a larger network topology with much more nodes and links. We find that it takes more timeslots for the agents to converge since more agents are engaged in routing decision and the input state space for GEANT is much larger than Abilene. However, DRL-OR shows similar performance to QoS-R and LBR on flow latency and throughput ratio, but provides much lower packet loss ratio for the latency-loss-sensitive flow.

D. Effectiveness Under Partial Deployment

To evaluate the potential of incremental deployment, we partially deploy the DRL-OR agents on a few important routing nodes and repeat the experiments. In particular, we try a simple case in which agents can only control the nodes adjacent to the bottleneck links. In this way we only need to deploy 2 agents in the Abilene scenario and 4 agents in the GEANT scenario. Note that in partial deployment we use the shortest path routing (SPR) as the fallback policy when routing loop occurs since we assume only part of routing nodes can be controlled by agents. The results of partially deployed DRL-OR are shown in Table III (i.e., DRL-OR part.). We find that only deploying agents to control the important routing nodes can improve performance significantly. Such a simple partial deployment design can achieve similar performance to the original DRL-OR for both light-load and heavy-load scenario in Abilene topology. In GEANT topology, many flows won't go through the nodes controlled by DRL agents. So the partial deployment seems less effective, but it still outperforms the fallback routing policy (i.e., SPR).

V. RELATED WORK

With the rapid progress of deep learning, machine learning (ML) technologies have been regarded as a promising solution to sophisticated network performance modeling and optimization problems, including TCP congestion control [32] and adaptive video streaming [33]. We focus on ML-based routing in this paper, and briefly classify existing studies based on the ML technology used.

Supervised learning for route generation: Zhuang *et al.* [34] propose a graph-aware deep learning-based algorithm for route generation in SDN-controlled scenario. Mao *et al.* [35] use deep belief network (DBN) to enable packet-level

TABLE III: Results of the performance metrics in different network scenarios.

Network scenario	Method	Avg. latency (ms)			Avg. thrpt. ratio (%)		Avg. loss ratio (%)
		Type I	Type III	Type IV	Type II	Type III	Type IV
Abilene light-load	SPR	20.06	20.95	21.39	96.59	96.51	2.02
	LBR	16.49	16.52	16.63	100.20	100.20	0.00
	QoS	11.47	11.51	11.65	100.21	98.90	1.59
	DRL-OR part.	11.85	11.99	12.37	100.02	99.59	0.12
	DRL-OR	11.71	11.83	12.22	99.95	99.62	0.06
Abilene heavy-load	SPR	142.57	145.08	150.95	80.31	80.03	17.18
	LBR	134.24	137.32	138.07	80.34	80.39	14.07
	QoS	150.06	131.00	152.65	81.75	81.30	17.14
	DRL-OR part.	100.29	101.05	98.46	83.22	82.64	13.35
	DRL-OR	91.49	96.09	99.78	83.70	83.86	12.98
GEANT	SPR	122.34	128.30	130.44	81.06	81.05	11.13
	LBR	17.43	17.76	17.52	96.54	96.43	3.26
	QoS	22.68	13.98	25.93	96.99	97.07	3.31
	DRL-OR part.	88.19	119.20	95.82	83.31	83.08	5.43
	DRL-OR	17.75	21.11	20.43	93.83	93.83	1.25

control and shortest path routing. Geyer *et al.* [36] propose a distributed routing algorithm using graph neural network (GNN) to learn the shortest path and max-min routing strategy. Kato *et al.* [37] indicate that generating or selecting route for the whole network needs a huge and complicated neural network, since the output space increases exponentially with the network topology growth. The routing approaches based on supervised learning can learn routing policies for specific networks, but the adaptiveness and the reliability under dynamically changing network status are not validated.

Reinforcement learning for network traffic optimization:

Many existing studies use Q-learning to design distributed routing algorithms to optimize specific objectives, including packet delay [38] [12], network lifetime [13], and transmission reliability [14]. The approaches proposed by these studies show good performance under specific scenarios. However, the Q-learning model is not capable of solving more complicated routing optimization problems. Recent studies use deep reinforcement learning to optimize network traffic. Xu *et al.* [15] leverage DDPG, an off-policy deep reinforcement learning approach, to generate flow split ratios given candidate routes of each flow so as to minimize the maximum link utilization. Valadarsky *et al.* [17] use TRPO to determine the best link weights, based on which the routing is computed. Xu *et al.* [16] study scalability and robustness issues of explicit path-based routing. They indicate that huge state and action spaces make it difficult to generate routes for the coming flows explicitly. Existing studies usually use deep reinforcement learning agent in a centralized manner, control the traffic load in an implicit way, which is lack of Interpretability and may not be suitable for complicated network optimization problem. Our approach proposed in this paper also leverages deep reinforcement learning, but we decompose the path generation process into hop-by-hop routing decisions. Thus, more complicated routing optimization problems can be tackled such as satisfying multi-type service requirements, and distributed and incremental deployment is enabled.

Safe online reinforcement learning: There are many researches on the development of safe reinforcement learning. Mao *et al.* [18] propose a safe online reinforcement learning

framework, called training wheels, to solve the load balance problem for web service requests. Miryoosefi *et al.* [39] aim at RL tasks with a wide range of constraints and present an algorithmic scheme for these tasks. Achiam *et al.* [40] propose constrained policy optimization for constrained RL. In this paper, we also introduce a safe online learning framework to make DRL-OR safe and reliable in the distributed routing scenarios.

VI. CONCLUSION

In this paper, we proposed DRL-OR, an online routing algorithm using multi-agent deep reinforcement learning for multiple service requirements. In DRL-OR scheme, each routing node is controlled by a DRL agent and the agent selects the best next hop for the coming flow requests quickly. To achieve such a scheme, we designed a general utility function with normalized performance metrics, modeled the route generation process as a multi-agent Markov decision process and developed an efficient learning algorithm, proposed a novel policy network structure, and provided a safe learning mechanism to enhance the reliability of the system. We implemented DRL-OR under SDN architecture and conducted experimental evaluations on Mininet using real-world network topologies and real traffic traces. Experiment results showed that DRL-OR with safe learning technique could well satisfy multiple types of service requirement and had great adaptiveness and reliability in different network scenarios. However, we also note that DRL-OR agents take more time to converge and learn a less optimal policy when state input space increases in large network topology. Using more efficient neural network structures like graph neural network (GNN) to extract network features will be a direction to explore in the future works.

ACKNOWLEDGMENT

This work is supported by the National Key R&D Program of China under Grant No. 2018YFB1800302, the National Natural Science Foundation of China under Grant (61625203, 61832013, 61872209, and 92038302), and the Independent Scientific Research Project of NUDT (ZZKYZX-03-02-02). Mingwei Xu and Yuan Yang are the corresponding authors. We thank the reviewers for their valuable comments.

REFERENCES

- [1] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "Rfc2475: An architecture for differentiated service," 1998.
- [2] C. Semeria, "Supporting differentiated service classes: queue scheduling disciplines," *Juniper networks*, pp. 11–14, 2001.
- [3] J. W. Guck, A. Van Bemten, M. Reisslein, and W. Kellerer, "Unicast qos routing algorithms for sdn: A comprehensive survey and performance evaluation," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 1, pp. 388–415, 2017.
- [4] S. Chen and K. Nahrstedt, "Distributed quality-of-service routing in high-speed networks based on selective probing," in *Proceedings of LCN*. IEEE, 1998, pp. 80–89.
- [5] Q. Ma and P. Steenkiste, "Supporting dynamic inter-class resource sharing: a multi-class qos routing algorithm," in *Proceedings of INFOCOM*, vol. 2. IEEE, 1999, pp. 649–660.
- [6] T. Korkmaz and M. Krunz, "Multi-constrained optimal path selection," in *Proceedings of INFOCOM*, vol. 2. IEEE, 2001, pp. 834–843.
- [7] M. Qin, K. Lei, B. Bai, and G. Zhang, "Towards a profiling view for unsupervised traffic classification by exploring the statistic features and link patterns," in *Proceedings of NetAI*. ACM, 2019, pp. 50–56.
- [8] T. T. Nguyen, G. Armitage, P. Branch, and S. Zander, "Timely and continuous machine-learning-based classification for interactive ip traffic," *IEEE/ACM TON*, vol. 20, no. 6, pp. 1880–1894, 2012.
- [9] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolk, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2014.
- [10] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izard, F. Mujica, and M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for sdn," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 99–110, 2013.
- [11] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [12] S. P. Choi and D.-Y. Yeung, "Predictive q-routing: A memory-based reinforcement learning approach to adaptive traffic control," in *Proceedings of NIPS*. MIT Press, 1996, pp. 945–951.
- [13] T. Hu and Y. Fei, "Qelar: A machine-learning-based adaptive routing protocol for energy-efficient and lifetime-extended underwater sensor networks," *IEEE TMC*, vol. 9, no. 6, pp. 796–809, 2010.
- [14] S. Basagni, V. Di Valerio, P. Gjanci, and C. Petrioli, "Marlin-q: Multimodal communications for reliable and low-latency underwater data delivery," *Elsevier Ad Hoc Networks*, vol. 82, pp. 134–145, 2019.
- [15] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang, "Experience-driven networking: A deep reinforcement learning based approach," in *Proceedings of INFOCOM*. IEEE, 2018, pp. 1871–1879.
- [16] Q. Xu, Y. Zhang, K. Wu, J. Wang, and K. Lu, "Evaluating and boosting reinforcement learning for intra-domain routing," in *MASS*. IEEE, 2019, pp. 265–273.
- [17] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of HotNets*. ACM, 2017, pp. 185–191.
- [18] H. Mao, M. Schwarzkopf, H. He, and M. Alizadeh, "Towards safe online reinforcement learning in computer systems," 2019.
- [19] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of HotNets*. ACM, 2010, p. 19.
- [20] K. Winstein and H. Balakrishnan, "Tcp ex machina: Computer-generated congestion control," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 123–134, 2013.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [22] K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Basar, "Fully decentralized multi-agent reinforcement learning with networked agents," in *Proceedings of ICML*. ACM, 2018, pp. 5867–5876.
- [23] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.
- [24] M. Xu, S. Yang, D. Wang, and J. Wu, "Efficient two dimensional-ip routing: An incremental deployment design," *Elsevier CN*, vol. 59, pp. 227–243, 2014.
- [25] H. Xu, X.-Y. Li, L. Huang, H. Deng, H. Huang, and H. Wang, "Incremental deployment and throughput maximization routing for a hybrid sdn," *IEEE/ACM TON*, vol. 25, no. 3, pp. 1861–1875, 2017.
- [26] A. Cianfrani, M. Listanti, and M. Polverini, "Incremental deployment of segment routing into an isp network: A traffic engineering perspective," *IEEE/ACM TON*, vol. 25, no. 5, pp. 3146–3160, 2017.
- [27] "Ryu," <https://github.com/osrg/ryu>, 2020.
- [28] I. Kostrikov, "Pytorch implementations of reinforcement learning algorithms," <https://github.com/ikostrikov/pytorch-a2c-ppo-acktr-gail>, 2018.
- [29] "Abilene," <http://www.cs.utexas.edu/~yzhang/research/AbileneTM/>, 2004.
- [30] S. Uhlig, B. Quoitin, J. Lepropre, and S. Balon, "Providing public intradomain traffic matrices to the research community," *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 1, pp. 83–86, 2006.
- [31] Z. Wang and J. Crowcroft, "Qos routing for supporting resource reservation," *IEEE JSAC*, 1996.
- [32] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on internet congestion control," in *Proceedings of ICML*. ACM, 2019, pp. 3050–3059.
- [33] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with pensieve," in *Proceedings of SIGCOMM*. ACM, 2017, pp. 197–210.
- [34] Z. Zhuang, J. Wang, Q. Qi, H. Sun, and J. Liao, "Graph-aware deep learning based intelligent routing strategy," in *Proceedings of LCN*. IEEE, 2018, pp. 441–444.
- [35] B. Mao, Z. M. Fadlullah, F. Tang, N. Kato, O. Akashi, T. Inoue, and K. Mizutani, "Routing or computing? the paradigm shift towards intelligent computer network packet transmission based on deep learning," *IEEE TC*, vol. 66, no. 11, pp. 1946–1960, 2017.
- [36] F. Geyer and G. Carle, "Learning and generating distributed routing protocols using graph-based deep learning," in *Proceedings of BigDAMA*. ACM, 2018, pp. 40–45.
- [37] N. Kato, Z. M. Fadlullah, B. Mao, F. Tang, O. Akashi, T. Inoue, and K. Mizutani, "The deep learning vision for heterogeneous network traffic control: Proposal, challenges, and future perspective," *IEEE wireless communications*, vol. 24, no. 3, pp. 146–153, 2016.
- [38] J. A. Boyan and M. L. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in *Proceedings of NIPS*. MIT Press, 1994, pp. 671–678.
- [39] S. Miryosefi, K. Brantley, H. Daume III, M. Dudik, and R. E. Schapire, "Reinforcement learning with convex constraints," in *Proceedings of NIPS*. MIT Press, 2019, pp. 14070–14079.
- [40] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proceedings of ICML*. ACM, 2017, pp. 22–31.